

INTERNSHIP PROJECT REPORT

On

"DataFlow Inc. Medallion Pipeline"

Submitted in partial fulfillment of the requirements of the Celebal Excellence Internship Program

at



Submitted By:

Kapil Singhal

JECRC University

Data Engineering Intern

Intern ID: CT_CSI_DE_548

Internship Duration: May 2026 - July 2026

Under the Guidance of

Ms. Yashashvi Dubey

Mr. Raj Biswas

(Mentors, Celebal Technologies)

Celebal Excellence Internship Program

Celebal Technologies

Session: 2026

1. Problem Statement

DataFlow Inc. , a growing data-driven organization, was struggling to answer basic data-related questions. Its data was scattered across raw CSV and JSON files - transactions, logs, product and customer data - with no central place to analyze it. The organization had no way to detect schema changes, no way to track data drift, and no process to ensure data quality. The objective was to build a centralized platform that ingests raw data, cleans it, detects structural changes, and maintains historical records for reliable analytics.

2. Solution & Approach

A Medallion Architecture pipeline was designed on Databricks, structured into three progressively refined data layers - Bronze, Silver, and Gold - supported by two cross-cutting reliability components: automated schema drift detection and SCD Type 2 history tracking. A SQL analytics layer was built on top for business-ready querying, and an Azure Data Factory pipeline was designed to orchestrate the entire flow.

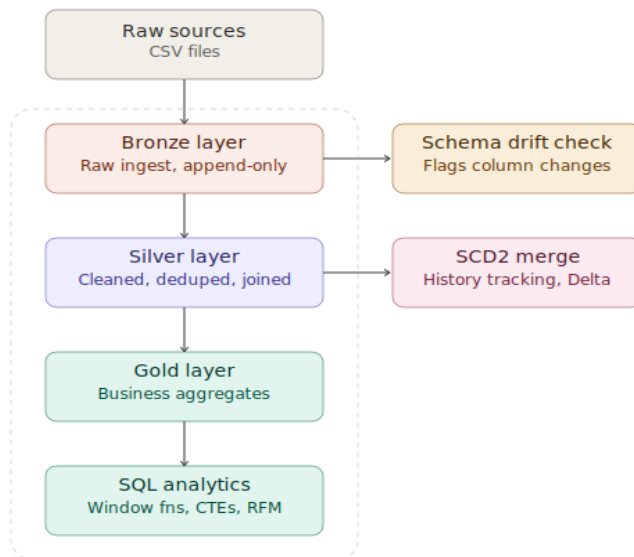


Fig 1: Pipeline architecture

Layer	Purpose	Technology
Bronze	Raw ingestion, append-only, schema-on-read	PySpark, Delta Lake
Silver	Cleaned, deduplicated, type-corrected, joined	PySpark
Gold	Business aggregates (sales, delivery, RFM, sellers)	PySpark
SCD Type 2	Dimension history tracking	Delta MERGE
Schema Drift	Detects schema changes between runs	PySpark + JSON
SQL Analytics	Window functions, CTEs, segmentation	Spark SQL
Orchestration	Sequenced pipeline execution	Azure Data Factory (designed)

3. Dataset

The Olist Brazilian E-Commerce public dataset was used, consisting of 9 relational CSV files covering the full lifecycle of an e-commerce order - customers, orders, order items, payments, reviews, products, sellers, geolocation, and category translations - totalling approximately 1.3 million rows.

File	Description	Rows
customers	Customer identifiers and location	99,441
orders	Order status and timestamps	99,441
order_items	Line items, price, freight	112,650
order_payments	Payment method and value	103,886
order_reviews	Review scores and text	99,224
products	Product attributes and category	32,951
sellers	Seller identifiers and location	3,095
geolocation	Zip code to lat/long mapping	1,000,163
category_translation	PT to EN category names	71

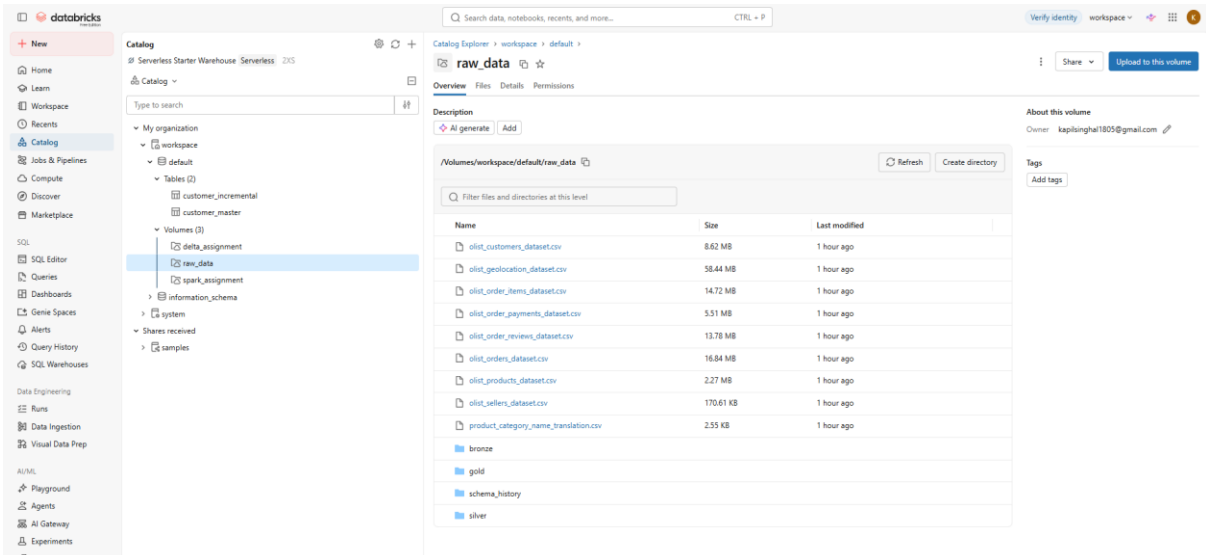


Fig 2: Raw datasets uploaded to a Unity Catalog Volume

4. Implementation

4.1 Bronze Layer - Raw Ingestion

All 9 raw CSV files are ingested as-is into Delta tables with no cleaning applied, each tagged with *_ingestion_timestamp* and *_source_file* for lineage. This preserves the original data so any downstream cleaning error can be traced back and reprocessed.

4.2 Silver Layer - Cleaning & Conforming

Bronze data is deduplicated on primary keys, timestamps are type-corrected, nulls are handled (median imputation for numeric fields, empty strings for missing text), zip codes are normalized, and category translations are joined in.

4.3 Gold Layer - Business Aggregates

Silver data is aggregated into business-ready tables: sales by category, revenue by state, seller performance, and customer RFM (Recency, Frequency, Monetary) metrics.

4.4 SCD Type 2 - History Tracking

Customer and product dimensions carry *effective_date*, *end_date*, and *is_current* columns. Delta Lake's MERGE INTO expires changed records and inserts new current versions, preserving history instead of overwriting it.

4.5 Schema Drift Detection

A script compares the current Bronze schema against the last recorded JSON snapshot on every run, flagging new, missing, or type-changed columns before they silently break downstream logic.

4.6 SQL Analytics Layer

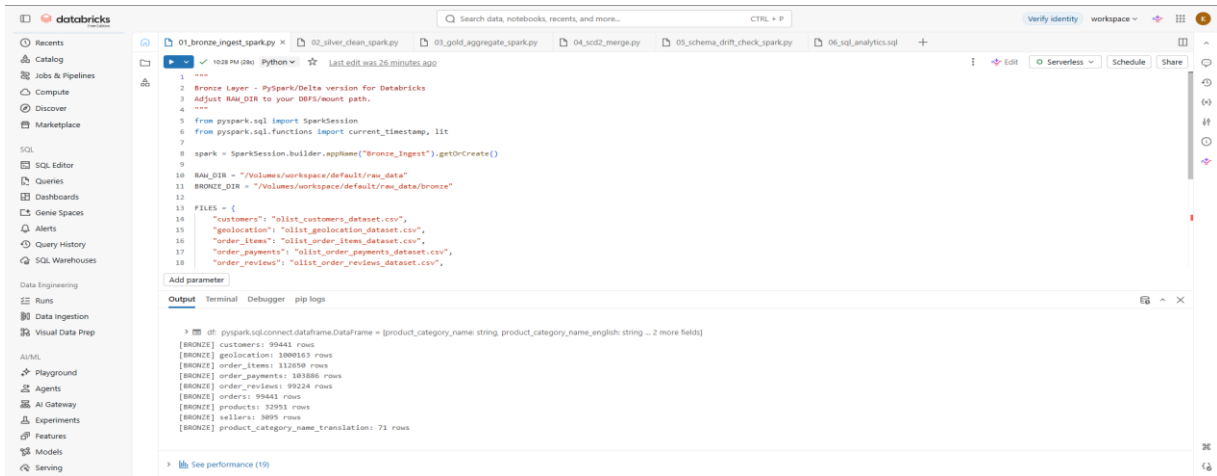
Built directly on Silver Delta tables using JOINS, GROUP BY, CASE, CTEs, and window functions (RANK, LAG, ROW_NUMBER OVER PARTITION BY) - covering category sales trends, delivery performance, seller rankings, and customer RFM segmentation (Loyal / Recent / At Risk / Churned).

4.7 Orchestration - Azure Data Factory

An ADF pipeline was designed with 5 Databricks Notebook activities chained via Succeeded dependency conditions: Bronze → Silver → Gold → SCD2 → Schema Drift Check, ensuring each stage only runs after the previous one completes successfully.

5. Execution Evidence

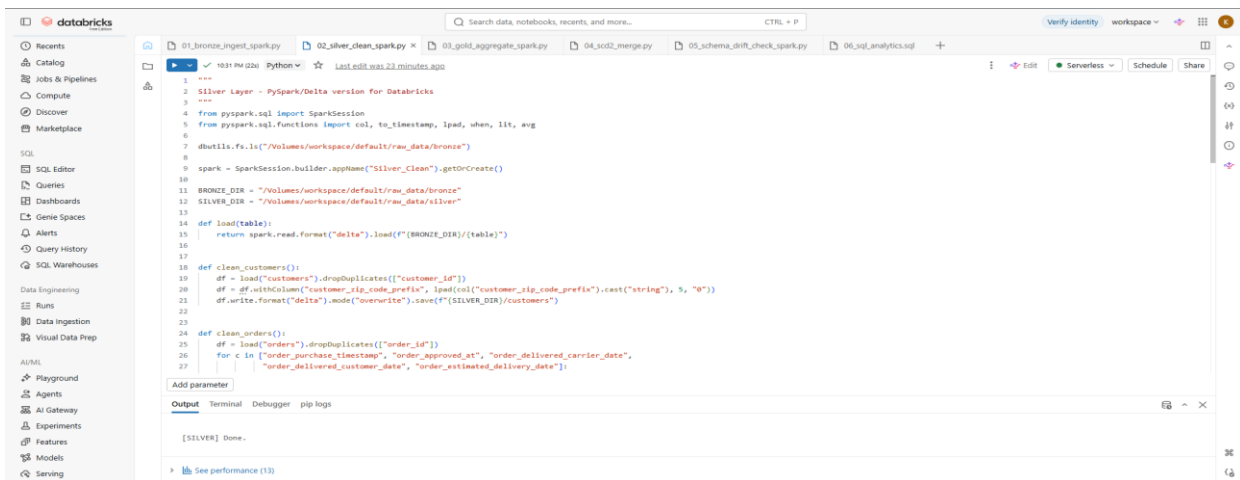
The pipeline was executed end-to-end on Databricks. Condensed notebook outputs are shown below.



The screenshot shows a Databricks notebook interface with a sidebar on the left containing navigation options like 'Recents', 'Catalog', 'Jobs & Pipelines', etc. The main area displays a Python notebook titled '01_bronze_ingest_spark.py'. The code defines a SparkSession, sets paths for BRONZE_DIR, and lists files to be loaded. The output section shows the results of the ingestion:

```
[BRONZE] customers: 99441 rows
[BRONZE] geolocation: 1808163 rows
[BRONZE] order_items: 112050 rows
[BRONZE] order_payments: 183886 rows
[BRONZE] order_reviews: 99224 rows
[BRONZE] orders: 99441 rows
[BRONZE] products: 32991 rows
[BRONZE] sellers: 3895 rows
[BRONZE] product_category_name_translation: 71 rows
```

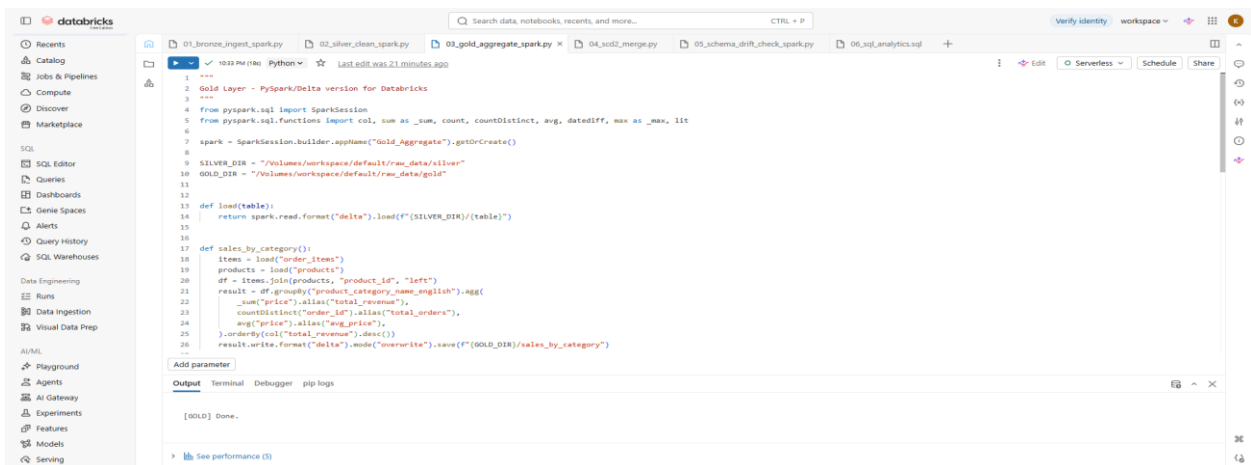
Bronze ingestion - all 9 tables loaded



The screenshot shows a Databricks notebook interface with a sidebar on the left. The main area displays a Python notebook titled '02_silver_clean_spark.py'. The code defines a SparkSession, sets paths for BRONZE_DIR and SILVER_DIR, and defines functions to load and clean data. The output section shows the results of the cleaning:

```
[SILVER] Done.
```

Silver cleaning completed



The screenshot shows a Databricks notebook interface with a sidebar on the left. The main area displays a Python notebook titled '03_gold_aggregate_spark.py'. The code defines a SparkSession, sets paths for SILVER_DIR and GOLD_DIR, and defines functions to load and aggregate data. The output section shows the results of the aggregation:

```
[GOLD] Done.
```

Gold aggregation completed

```

1 ---
2 SCD Type 2 - History Tracking on Delta Lake (Databricks)
3 Tracks changes on customers & products dimensions using MERGE INTO.
4 Run this in Databricks notebook / job.
5 ---
6 from pyspark.sql import SparkSession
7 from pyspark.sql.functions import col, current_timestamp, lit
8 from delta.tables import DeltaTable
9
10 spark = SparkSession.builder.appName("SCD2_Merge").getOrCreate()
11
12 SILVER_PATH = "/Volumes/workspace/default/raw_data/silver"
13 GOLD_PATH = "/Volumes/workspace/default/raw_data/gold/dim"
14
15
16 def init_scd2_table(df, key_col, path):
17     """First-time load: add SCD2 tracking columns."""
18     df = df.withColumn("effective_date", current_timestamp()) \
19         .withColumn("end_date", lit(None).cast("timestamp")) \
20         .withColumn("is_current", lit(True))
21     df.write.format("delta").mode("overwrite").save(path)
22
23
24 def scd2_merge(new_df, key_col, path, compare_cols):
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

SCD Type 2 merge executed

```

1 ---
2 Schema Drift Detection - PySpark/Delta version for Databricks
3 ---
4 Import json
5 from pyspark.sql import SparkSession
6
7 spark = SparkSession.builder.appName("Schema_Drift").getOrCreate()
8
9 BRONZE_DIR = "/Volumes/workspace/default/raw_data/bronze"
10 SCHEMA_LOG_DIR = "/Volumes/workspace/default/raw_data/schema_history"
11
12 TABLES = ["customers", "geo_location", "order_items", "order_payments",
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Schema drift baseline recorded

```

1 --
2 -- SQL Layer
3 --
4
5 -- 1. Sales by category with running total
6 WITH category_sales AS (
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

SQL analytics - RFM segmentation output

6. Tech Stack Coverage

Skill	Applied In
Python / Data Structures / Pandas	All notebooks, initial prototyping
SELECT, WHERE, GROUP BY, JOINS	SQL analytics layer, Silver/Gold joins
CASE, CTE, Window Functions	SQL analytics - segmentation, ranking, trends
ETL Concepts	Bronze → Silver → Gold pipeline design
ADF Pipelines / Orchestration	adf_pipeline.json - designed and documented
Spark Basics / DataFrames	All PySpark notebooks
Read/Write CSV/JSON, Transformations	Bronze ingestion, Silver cleaning
Delta Lake / Versioning	All layers stored as Delta tables
Data Modeling	Gold layer aggregate design
SCD + Merge	SCD2 notebook - Delta MERGE INTO
Medallion (Bronze/Silver/Gold)	Overall pipeline architecture

7. Challenges Faced

Free-tier compute limitation. Databricks Free Edition runs Serverless compute only, with no persistent Cluster ID. Azure Data Factory's Databricks linked service requires either an existing cluster or quota to provision a job cluster - neither was available, blocking live ADF execution.

CSV multi-line parsing. The order reviews file contains free-text comments with embedded line breaks, which broke default CSV parsing and shifted columns. Resolved using Spark's multiLine, quote, and escape CSV options.

Delta schema merge conflicts. Re-running Bronze after a partial run caused schema merge and duplicate column errors from stale metadata. Resolved by clearing the Bronze path before a clean re-run and dropping duplicate lineage columns before joins.

Unity Catalog Volumes vs local filesystem. Standard Python file checks (os.path.exists) do not work against Volume paths. Replaced with dbutils.fs.ls()-based existence checks in the SCD2 merge and schema drift scripts.

SQL table registration on Volumes. CREATE TABLE ... LOCATION failed with file-scheme errors on Unity Catalog Volumes. Worked around by querying Delta tables directly by path instead of registering managed tables.

Azure-Databricks linked service setup. Configuring the ADF-to-Databricks connection (workspace domain, PAT token, cluster selection) required troubleshooting several authentication and configuration steps as a first-time Azure user.

8. Known Limitations

The ADF pipeline was fully designed and configured (linked service, activities, dependency chaining) but could not be executed end-to-end live, since Databricks Free Edition and free-tier Azure subscriptions do not provide the compute quota required for ADF-managed job clusters. The pipeline definition in *adf_pipeline.json* reflects exactly what would run unchanged on a standard, paid Azure Databricks workspace with Jobs/Workflows enabled. All 5 pipeline stages were independently validated by running the notebooks directly in Databricks.

9. Future Scope

- Execute the ADF pipeline live on a paid Azure Databricks workspace with job cluster support.
- Build a BI dashboard (Power BI or Databricks SQL Dashboard) on top of the Gold layer tables.
- Extend SCD Type 2 tracking to additional dimensions (sellers, order status).
- Add automated data quality tests (row count checks, null thresholds, referential integrity) as a dedicated validation layer.
- Add CI/CD for notebook deployment using Databricks Asset Bundles.

10. Conclusion

This project delivers a working, end-to-end solution to DataFlow Inc.'s original problem: raw, scattered data with no schema visibility, no quality checks, and no history is now processed through a structured Medallion pipeline that cleans it, tracks structural changes automatically, preserves historical dimension data, and exposes clean, query-ready analytics.

Full source code, SQL scripts, the ADF pipeline definition, and setup documentation are available at: github.com/KapilSinghal21/DATAFLOW-INC-MEDALLION.